INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY (ICDFA)

Case Study 3 - Investigating Memory Evidence (Volatility 2)

Lab Title:	**Investigating Memory Evidence**
Student Name:	**Yusuf Oluwaseun Dada**
Student ID:	**2025/FWSD/11339**
Course Code:	**SBT-DF204**
Course Name:	**Computer Forensics Case Studies**

Instructor: Mr. Aminu Idris, AMCPN

Date: 19/10/2026

1. Required Tools and Environment

Item	Recommended Tool	Purpose
Operating system	ICDFA Kali Linux VM	Run the analysis.
Memory forensics	Volatility 2 (python2 vol.py)	Parse the memory image.
Hashing	sha256sum	Preserve evidence integrity.
Evidence	memdumpWin7.mem	Supplied memory image.
Viewing	Web browser (local file only)	Open the shellbags HTML output.

2. Case Folder Setup and Acquire the Image

```
┌──(kali㉿kali)-[~]
└─$ mkdir -p ~/SBT-DF204-Case3/{evidence,working,exported,reports,screenshots}
cd ~/SBT-DF204-Case3
pwd
df -h .
/home/kali/SBT-DF204-Case3
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        79G   28G   47G  38% /
```

```
┌──(kali㉿kali)-[~/SBT-DF204-Case3]
└─$ git clone https://github.com/volatilityfoundation/volatility.git
Cloning into 'volatility' ...
remote: Enumerating objects: 27414, done.
remote: Counting objects: 100% (2/2), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 27414 (delta 0), reused 0 (delta 0), pack-reused 27412 (from 2)
Receiving objects: 100% (27414/27414), 21.11 MiB | 248.00 KiB/s, done.
Resolving deltas: 100% (19758/19758), done.
```

```
ls
python2 vol.py --info | head -n 20
python2 --version
AUTHORS.txt    contrib      LEGAL.txt    Makefile     PKG-INFO     pyinstaller.spec  resources  tools       vol.py
CHANGELOG.txt  CREDITS.txt  LICENSE.txt  MANIFEST.in  pyinstaller  README.txt        setup.py   volatility
Volatility Foundation Volatility Framework 2.6.1
*** Failed to import volatility.plugins.registry.shutdown (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.getservicesids (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.timeliner (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.malware.apihooks (NameError: name 'distorm3' is not defined)
*** Failed to import volatility.plugins.malware.servicediff (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.registry.userassist (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.getsids (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.registry.shellbags (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.evtlogs (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.registry.shimcache (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.tcaudit (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.registry.dumpregistry (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.registry.lsadump (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.malware.threads (NameError: name 'distorm3' is not defined)
*** Failed to import volatility.plugins.mac.apihooks_kernel (ImportError: No module named distorm3)
*** Failed to import volatility.plugins.registry.amcache (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.mac.check_syscall_shadow (ImportError: No module named distorm3)
*** Failed to import volatility.plugins.malware.svcscan (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.registry.auditpol (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.ssdt (NameError: name 'distorm3' is not defined)
Traceback (most recent call last):
  File "vol.py", line 192, in <module>
    main()
  File "vol.py", line 152, in main
```

Activate

```
Traceback (most recent call last):
  File "vol.py", line 192, in <module>
    main()
  File "vol.py", line 152, in main
    print_info()
  File "vol.py", line 133, in print_info
    print "{0:{2}} - {1:15}".format(name, doc, max_length)
IOError: [Errno 32] Broken pipe
Python 2.7.18
```

```
--2026-10-19 19:22:57--  (try: 2)  https://uc1eab511dd8c289d7bb6f53b63c.dl.dropboxusercontent.com/cd/0/inline2/DKWPmrBogW9vty_u7ZeQdO4dSbF
2f9aqIUadslkgJ5eOUI1AgC6A_HEZeAUyyWoaGGfQBw85GYn0Y0o4lds0QFrhui_a9IiAKbOKSCJA6d9rdpUmKkM0TYpdTQezalnVxSp9s_lFYiPtuGn0m-ImuMZ_MfojYpZmpHNz2
s9q1AlcM-27XVXLjphf8sYnHuhpu0TkC7M6l7Y4QX3DVNzANw2RyuwWyhXZxD-EQFzYmeIVo627BQ9pNoxIuPZjsr9tW3EpFfA3zcJyrKa-oDeXG1rIo_MdFPNkFT3LETuF-6Y4Q-W
E9K7qfszXOIknLQrXxi99OQGDzzdoEB4pg7D_7_RhbSS9ujYHLnSusdmE3GbPYA/file?dl=1
Connecting to uc1eab511dd8c289d7bb6f53b63c.dl.dropboxusercontent.com (uc1eab511dd8c289d7bb6f53b63c.dl.dropboxusercontent.com)|162.125.64.1
5|:443... connected.
HTTP request sent, awaiting response... 206 Partial Content
Length: 1073676288 (1024M), 26918912 (26M) remaining [application/binary]
Saving to: 'evidence/memdumpWin7.mem'

evidence/memdumpWin7.mem          100%[++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++=>]   1024M   820KB/s    in 40s

2026-10-19 19:23:38 (663 KB/s) - 'evidence/memdumpWin7.mem' saved [1073676288/1073676288]
```

```
┌──(kali㉿kali)-[~/SBT-DF204-Case3]
└─$ ls -la evidence/memdumpWin7.mem
file evidence/memdumpWin7.mem
head -c 200 evidence/memdumpWin7.mem
-rw-rw-r-- 1 kali kali 1073676288 Oct 19 19:23 evidence/memdumpWin7.mem
evidence/memdumpWin7.mem: data
```

```
┌──(kali㉿kali)-[~]
└─$ cd SBT-DF204-Case3

┌──(kali㉿kali)-[~/SBT-DF204-Case3]
└─$ sha256sum evidence/memdumpWin7.mem | tee reports/memdump_sha256.txt
cp --preserve=timestamps evidence/memdumpWin7.mem working/memdumpWin7_working.mem
sha256sum working/memdumpWin7_working.mem | tee -a reports/memdump_sha256.txt
^[[5~500bc1ba8b5ffa8053b6213d7e7aae0c84c18cce541dc53a55887e6e5099495e  evidence/memdumpWin7.mem
^[[5~^[[5~^[[5~^[[5~500bc1ba8b5ffa8053b6213d7e7aae0c84c18cce541dc53a55887e6e5099495e  working/memdumpWin7_working.mem

┌──(kali㉿kali)-[~/SBT-DF204-Case3]
└─$ chmod a-w evidence/memdumpWin7.mem
```

I Placed the supplied image into evidence/ as my untouched original, then fingerprints it with SHA-256. cp makes the working copy that every Volatility command will read, leaving the original byte-for-byte preserved. Matching hashes prove the copy is identical. chmod a-w removes write permission from the original as a precaution against accidental modification

3. Evidence Acquisition and Integrity

Field	Entry
Original file name	memdumpWin7.mem
Source	Downloaded via wget from Dropbox link provided in course lab materials (https://www.dropbox.com/s/x41f2lyhlrixts6/memdumpWin7.mem)
File size (bytes)	1,073,676,288 bytes (1 GiB)
SHA-256 (original)	500bc1ba8b5ffa8053b6213d7e7aae0c84c18cce541dc53a55887e6e5099495e
SHA-256 (working copy)	500bc1ba8b5ffa8053b6213d7e7aae0c84c18cce541dc53a55887e6e5099495e (matches original exactly)
Working-copy method	cp --preserve=timestamps; hashes compared and confirmed identical; original set read-only via chmod a-w (confirmed: -r--r--r--)
Tools and versions	Volatility 2, version 2.6.1 (python2 vol.py); Python 2.7.18
Analysis workstation	Kali

4. Part A - Identify the Profile and Image Time

```
----------------------------------
Registry: \REGISTRY\MACHINE\BCD00000000
Key name: NewStoreRoot (S)
Last updated: 2019-01-06 18:02:44 UTC+0000

Subkeys:
  (S) Description
  (S) Objects

Values:
----------------------------------
Registry: \??\C:\Users\IEUser\ntuser.dat
Key name: CMI-CreateHive{6A1C4018-979D-4291-A7DC-7AED1C75B67C} (S)
Last updated: 2019-01-06 15:02:48 UTC+0000

Subkeys:
  (S) AppEvents
  (S) Console
  (S) Control Panel
  (S) Environment
  (S) EUDC
  (S) Keyboard Layout
  (S) Network
  (S) Printers
  (S) Software
  (S) System
  (V) Volatile Environment

Values:
----------------------------------
```

cd ~/SBT-DF204-Case3/volatility
python2 vol.py -f ~/SBT-DF204-Case3/working/memdumpWin7_working.mem --profile=Win7SP1x86_23418 printkey

```
----------------------------------
Registry: \??\C:\Users\IEUser\ntuser.dat
Key name: Volatile Environment (V)
Last updated: 2019-01-06 15:02:48 UTC+0000

Subkeys:
  (V) 1

Values:
REG_SZ        LOGONSERVER     : (V) \\IE8WIN7
REG_SZ        USERDOMAIN      : (V) IE8WIN7
REG_SZ        USERNAME        : (V) IEUser
REG_SZ        USERPROFILE     : (V) C:\Users\IEUser
REG_SZ        HOMEPATH        : (V) \Users\IEUser
REG_SZ        HOMEDRIVE       : (V) C:
REG_SZ        APPDATA         : (V) C:\Users\IEUser\AppData\Roaming
REG_SZ        LOCALAPPDATA    : (V) C:\Users\IEUser\AppData\Local
```

cd ~/SBT-DF204-Case3/volatility

python2 vol.py -f ~/SBT-DF204-Case3/working/memdumpWin7_working.mem --profile=Win7SP1x86_23418 printkey -K 'Volatile Environment'

```
Registry: \REGISTRY\MACHINE\SYSTEM
Key name: TimeZoneInformation (S)
Last updated: 2019-01-06 16:31:39 UTC+0000

Subkeys:

Values:
REG_DWORD     Bias            : (S) 480
REG_DWORD     DaylightBias    : (S) 4294967236
REG_SZ        DaylightName    : (S) @tzres.dll,-211
REG_BINARY    DaylightStart   : (S)
0×00000000  00 00 03 00 02 00 02 00 00 00 00 00 00 00 00 00   ................
REG_DWORD     StandardBias    : (S) 0
REG_SZ        StandardName    : (S) @tzres.dll,-212
REG_BINARY    StandardStart   : (S)
0×00000000  00 00 0b 00 01 00 02 00 00 00 00 00 00 00 00 00   ................
REG_SZ        TimeZoneKeyName : (S) Pacific Standard Time
REG_DWORD     DynamicDaylightTimeDisabled : (S) 0
REG_DWORD     ActiveTimeBias  : (S) 480
```

IMG=~/SBT-DF204-Case3/working/memdumpWin7_working.mem
python2 vol.py -f "$IMG" --profile=Win7SP1x86_23418 printkey -K 'ControlSet001\Control\TimeZoneInformation' | tee ~/SBT-DF204-Case3/reports/timezone.txt

It reads the time-zone settings stored in the SYSTEM registry hive in memory, giving evidence of the machine's configured zone to compare against the difference between the UTC and local times that imageinfo reported.

```
┌──(kali㉿kali)-[~/SBT-DF204-Case3/volatility]
└─$ IMG=~/SBT-DF204-Case3/working/memdumpWin7_working.mem
python2 vol.py -f "$IMG" --profile=Win7SP1x86_23418 pslist > ~/SBT-DF204-Case3/reports/pslist.txt 2>&1
python2 vol.py -f "$IMG" --profile=Win7SP1x86_23418 netscan > ~/SBT-DF204-Case3/reports/netscan.txt 2>&1
```

IMG=~/SBT-DF204-Case3/working/memdumpWin7_working.mem
python2 vol.py -f "$IMG" --profile=Win7SP1x86_23418 pslist > ~/SBT-DF204-Case3/reports/pslist.txt 2>&1
python2 vol.py -f "$IMG" --profile=Win7SP1x86_23418 netscan > ~/SBT-DF204-Case3/reports/netscan.txt 2>&1

```
┌──(kali㉿kali)-[~/SBT-DF204-Case3/volatility]
└─$ cat ~/SBT-DF204-Case3/reports/pslist.txt | grep -i -E "System|smss|csrss|wininit|services|lsass|explorer"
*** Failed to import volatility.plugins.getservicesids (ImportError: No module named Crypto.Hash)
0×8413a908 System                    4      0     73      518 ------      0 2019-01-06 18:02:44 UTC+0000
0×851ec488 smss.exe                236      4      2       29 ------      0 2019-01-06 18:02:44 UTC+0000
0×852b11e0 csrss.exe               300    292      8      434      0      0 2019-01-06 18:02:44 UTC+0000
0×841c32b8 wininit.exe             340    292      3       75      0      0 2019-01-06 18:02:44 UTC+0000
0×852b21d0 csrss.exe               348    328      9      316      1      0 2019-01-06 18:02:44 UTC+0000
0×858b0310 services.exe            432    340     10      228      0      0 2019-01-06 18:02:45 UTC+0000
0×858bbd28 lsass.exe               440    340      7      581      0      0 2019-01-06 18:02:45 UTC+0000
0×85bba860 explorer.exe           2524   2500     32      952      1      0 2019-01-06 15:02:54 UTC+0000
```

cat ~/SBT-DF204-Case3/reports/pslist.txt | grep -i -E "System|smss|csrss|wininit|services|lsass|explorer"

The imageinfo inspects the image and suggests which memory profile (operating system and service pack) matches it. It also reports the image date and time, in both UTC and local time, which is the moment the memory was captured. It can takes few minutes. The output lists several 'Suggested Profile(s)', as the are candidates.

Item	Findings
Suggested profile(s) from imageinfo	Win7SP1x86_23418, Win7SP0x86, Win7SP1x86_24000, Win7SP1x86
Profile selected	Win7SP1x86_23418
How it was validated (which plugins gave clean output)	pslist gave a clean, correctly-ordered process list; netscan gave a valid listening socket record
Operating system actually indicated (note the Win7 vs Win10 discrepancy)	Windows 7 SP1, 32-bit (x86), contradicts the scenario's "Windows 10" wording
Image date/time (UTC)	2019-01-06 15:09:06 UTC+0000
Image local date/time and offset	2019-01-06 07:09:06 -0800 (UTC-8)
Time zone per registry	Pacific Standard Time (Bias 480 / UTC-8), confirmed via TimeZoneInformation key, last updated 2019-01-06 16:31:39 UTC (~82 min after the image capture time, noted as a minor timing observation, not treated as inconsistent)

5. Part B - User and Account Artefacts

```
Registry: \SystemRoot\System32\Config\SOFTWARE
Key name: ProfileList (S)
Last updated: 2015-09-21 09:50:52 UTC+0000

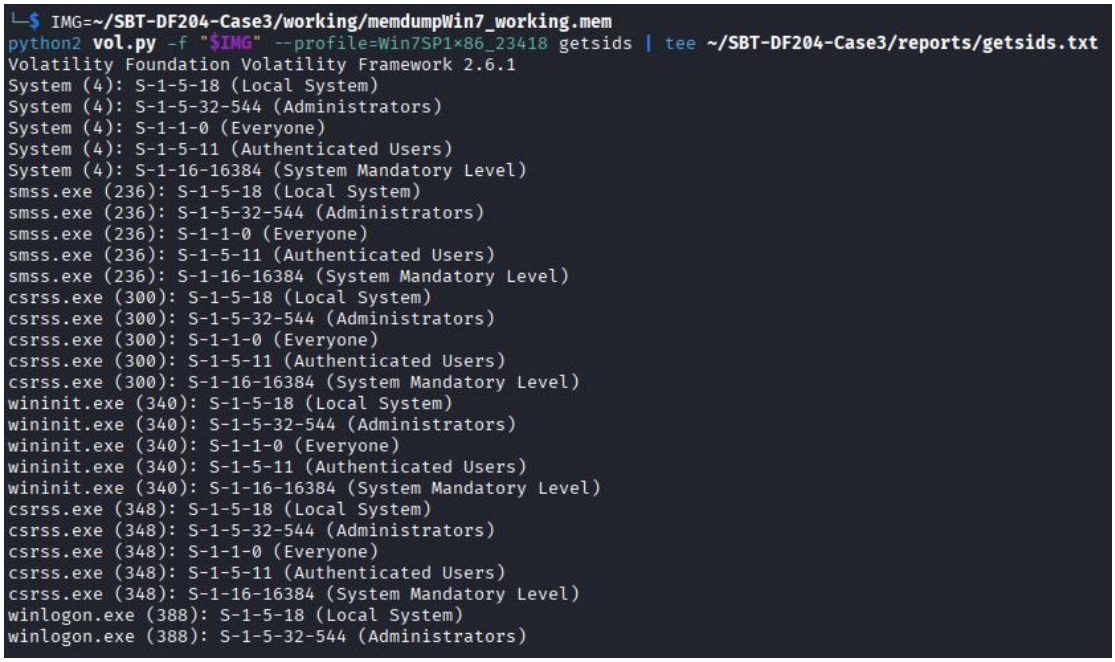
Subkeys:
  (S) S-1-5-18
  (S) S-1-5-19
  (S) S-1-5-20
  (S) S-1-5-21-1716914095-909560446-1177810406-1000
  (S) S-1-5-21-1716914095-909560446-1177810406-1002

Values:
REG_EXPAND_SZ ProfilesDirectory : (S) %SystemDrive%\Users
REG_EXPAND_SZ Default           : (S) %SystemDrive%\Users\Default
REG_EXPAND_SZ Public            : (S) %SystemDrive%\Users\Public
REG_EXPAND_SZ ProgramData       : (S) %SystemDrive%\ProgramData
Volatility Foundation Volatility Framework 2.6.1
```

```
Virtual    Physical   Name
---------- ---------- ----
0x87c10370 0x27fd5370 [no name]
0x87c1c008 0x27fa3008 \REGISTRY\MACHINE\SYSTEM
0x87c459c8 0x27d8e9c8 \REGISTRY\MACHINE\HARDWARE
0x889c0430 0x1285a430 \??\C:\Windows\ServiceProfiles\NetworkService\NTUSER.DAT
0x8b46d008 0x16d8c008 \??\C:\Windows\ServiceProfiles\LocalService\NTUSER.DAT
0x8b5d39b0 0x120049b0 \??\C:\Users\IEUser\ntuser.dat
0x97104008 0x1c45f008 \SystemRoot\System32\Config\SECURITY
0x9711d9c8 0x1e6a09c8 \SystemRoot\System32\Config\SOFTWARE
0x981f9008 0x1b446008 \SystemRoot\System32\Config\DEFAULT
0x98209518 0x1f3a7518 \SystemRoot\System32\Config\SAM
0x982d55c0 0x1e17c5c0 \REGISTRY\MACHINE\BCD00000000
0xa620b008 0x1997b008 \??\C:\Users\IEUser\AppData\Local\Microsoft\Windows\UsrClass.dat
0xa65269c8 0x17ad09c8 \??\C:\Users\sshd_server\ntuser.dat
0xa6539260 0x0f690260 \??\C:\Users\sshd_server\AppData\Local\Microsoft\Windows\UsrClass.dat
0xa7d2a9c8 0x01c469c8 \??\C:\System Volume Information\Syscache.hve
Volatility Foundation Volatility Framework 2.6.1
```

IMG=~/SBT-DF204-Case3/working/memdumpWin7_working.mem
python2 vol.py -f "$IMG" --profile=Win7SP1x86_23418 hivelist | tee ~/SBT-DF204-Case3/reports/hivelist.txt
python2 vol.py -f "$IMG" --profile=Win7SP1x86_23418 printkey -K 'Microsoft\Windows NT\CurrentVersion\ProfileList' | tee ~/SBT-DF204-Case3/reports/profilelist.txt
python2 vol.py -f "$IMG" --profile=Win7SP1x86_23418 getsids | tee ~/SBT-DF204-Case3/reports/getsids.txt

hivelist shows which registry hives (SYSTEM, SOFTWARE, SAM, and each user's NTUSER.DAT) are loaded in memory and their virtual and physical offsets. The ProfileList key (in the SOFTWARE hive) maps each account SID to its profile folder path, such as C:\Users\Name, to know which user profiles exist on the machine. getsids lists the SIDs that each running process belongs to, which links processes (like a browser or cmd.exe) to an account.

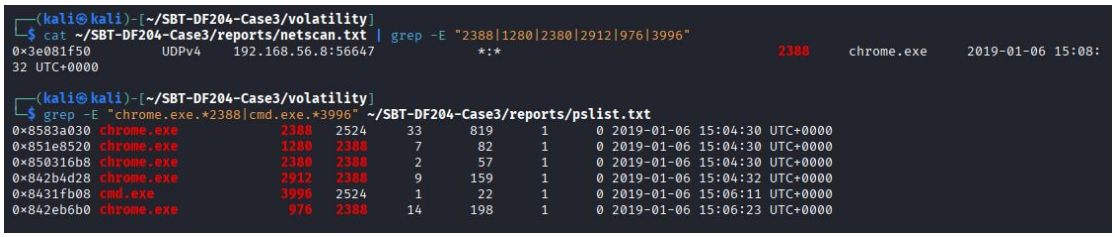


IMG=~/SBT-DF204-Case3/working/memdumpWin7_working.mem
python2 vol.py -f "$IMG" --profile=Win7SP1x86_23418 getsids | tee ~/SBT-DF204-Case3/reports/getsids.txt

Item	Findings
User profile(s) found and profile paths	C:\Users\IEUser\ntuser.dat and C:\Users\sshd_server\ntuser.dat
SID(s) observed	S-1-5-21-1716914095-909560446-1177810406-**1000** = IEUser; S-1-5-21-1716914095-909560446-1177810406-**1002** = sshd_server
NTUSER.DAT hive virtual offset(s)	IEUser: 0x8b5d39b0 ; sshd_server: 0xa65269c8
Which SID owns the key processes (browser, cmd.exe, explorer.exe)	IEUser (SID ...-1000) owns explorer.exe (2524), four chrome.exe instances (2388, 1280, 2380, 2912, 976), cmd.exe (3996), conhost.exe (3988), and FTK Imager.exe (2596). sshd_server (SID ...-1002) owns only SSH-related service infrastructure (cygrunsrv.exe, sshd.exe), not implicated in the browser/command/transfer activity
Limit between profile evidence and a person at the keyboard	getsids shows which account each process ran under, it does not identify which human was physically at the keyboard when IEUser's session was active. IEUser is also an Administrators-group local account (visible via the S-1-5-32-544 group membership listed

Item	Findings
	for its processes), meaning privilege alone doesn't narrow this to one specific individual either

6. Part C - Process and Network Evidence



PID	Process	Parent PID	Start (UTC)	Relevant connection (local → foreign, state)	Observation / limit
2388	chrome.exe	2524 (explorer.exe)	15:04:30	UDPv4 192.168.56.8:56647, bound locally	Main Chrome process, launched by the user's desktop shell. The UDP binding is normal Chrome background behavior, not evidence of a specific transfer on its own.
1280	cmd.exe	2388 (chrome.exe)	15:04:30	None found	Chrome child process (renderer/tab), standard multi-process architecture
2380	chrome.exe	2388 (chrome.exe)	15:04:30	None found	Chrome child process
976	chrome.exe	2388 (chrome.exe)	15:06:23	None found	Chrome child process, started ~12 seconds after cmd.exe

7. Part D - Command History



IMG=~/SBT-DF204-Case3/working/memdumpWin7_working.mem
python2 vol.py -f "$IMG" --profile=Win7SP1x86_23418 cmdscan | tee ~/SBT-DF204-Case3/reports/cmdscan.txt
python2 vol.py -f "$IMG" --profile=Win7SP1x86_23418 consoles | tee ~/SBT-DF204-Case3/reports/consoles.txt

Item	Findings
Console process and PID	conhost.exe (PID 3988), attached to cmd.exe (PID 3996)
Relevant command(s) exactly as recorded	ipconfig; cd Downloads; copy secret_file.docx F:
Path or drive references (source and destination)	Source: C:\Users\IEUser\Downloads\secret_file.docx ; Destination: F: (drive letter, not yet confirmed as removable
Output showing success or failure, if any	Console screen buffer directly shows “1 file(s) copied.” a genuine success confirmation from the shell, not merely an inferred command
Timing or correlation evidence	ipconfig’s own output confirms the host IP (192.168.56.8) matches the Chrome UDP binding found in Part C’s netscan, corroborating this is the same live session; the Windows version banner also confirms Windows 7 (Version 6.1.7601), directly corroborating Part A’s profile selection

Item	Findings
Can completion be proved? State the limit	Within the shell session itself, yes, the "1 file(s) copied" message is direct evidence the copy command completed successfully inside Windows. However, this does not by itself prove drive F: was removable media (that requires Part E correlation via USBSTOR/MountedDevices), nor does it prove what happened to the file afterward (whether it was removed from the device, transferred elsewhere, or remained there)

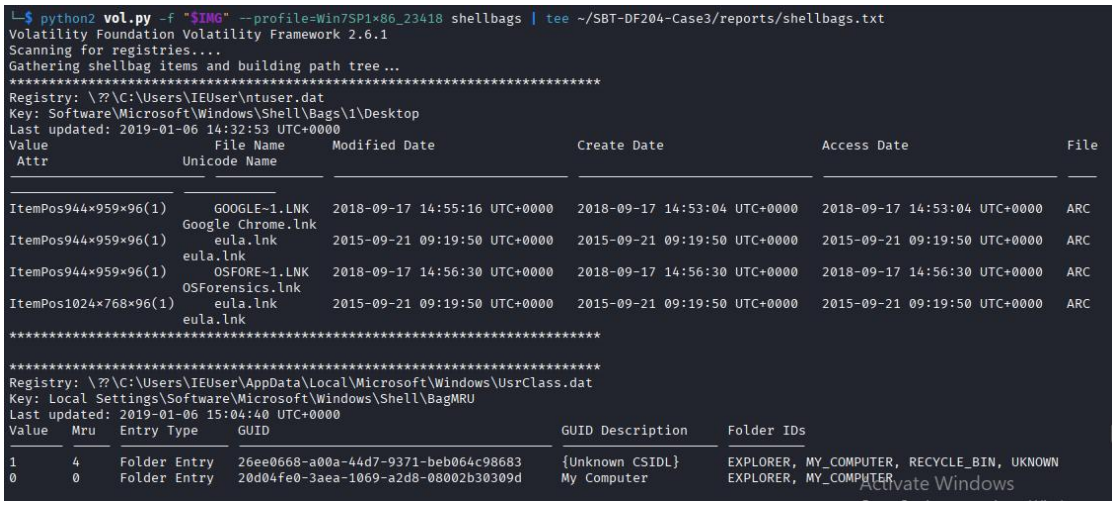
8. Part E - Removable-Media and Drive-Mapping Evidence



Artefact	Key / value observed	Last-write (UTC)	What it establishes	What it does NOT establish
USBSTOR	Disk&Ven_General& Prod_UDisk&Rev_5.00, serial 6&1bec0f48&0&	15:02:54	A generic USB flash drive connected to the system	Possession, user identity, files copied
MountedDevices	\DosDevices\F: → same USBSTOR serial	15:02:54	F: drive letter assigned to this exact USB device	Timing relative to the copy command specifically
DeviceClasses	{53f56307-...} subkey confirms same USBSTOR serial	15:02:54	Third independent registry confirmation of the same device	Nothing new beyond corroboration
MountPoints2	Subkey "F" under IEUser's own NTUSER.DAT	15:03:07	**IEUser's own user profile recorded**	Still does not prove which human was

Artefact	Key / value observed	Last-write (UTC)	What it establishes	What it does NOT establish
			accessing drive F: the person-level link between this specific device and this specific account	using the IEUser account at that exact second

9. Part F - File Explorer Evidence (ShellBags)



python2 vol.py -f "$IMG" --profile=Win7SP1x86_23418 shellbags | tee ~/SBT-DF204-Case3/reports/shellbags.txt

Item	Findings
Downloads folder entry and times	C:\Users\IEUser\Downloads, ShellBag last-write 15:04:40 UTC; folder's own modified/access time 14:49:40 UTC
Folder containing the document of interest	Downloads (directly matches the cd Downloads)
Removable-drive path entry and times	F:\ volume entry, ShellBag last-write 15:07:03 UTC (after the copy command)
Which registry hive/user the entries came from	IEUser's own UsrClass.dat (Local Settings\Software\Microsoft\Windows\Shell\BagMRU), fourth independent IEUser-specific artefact, alongside MountPoints2
Folder-view evidence vs. file-modification evidence	The Downloads folder's listed "Modified Date" (14:49:40) reflects the folder's own last content change, consistent with, but not conclusive proof of,

Item	Findings
	secret_file.docx being placed there at that time. ShellBag last-write times (15:04:40, 15:07:03) reflect when Explorer's view/preference data was recorded, not necessarily the exact second of each file action

10. Timeline

Stage	UTC Timestamp	Evidence ID	Source Artefact	Observed Fact	Interpretation and Limit
Browser or process evidence	15:04:30	E02	pslist	chrome.exe (PID 2388) starts, parent explorer.exe (PID 2524)	Shows browser launched under IE interactive session shortly after US connection
Download-folder review	14:49:40	E07	ShellBags (BagMRU)	Downloads folder's own modified/access time	Consistent with secret_file.docx b placed in Downloads around this t direct file-level timestamp for that file
Removable-device evidence	15:02:54	E04, E05	USBSTOR, MountedDevices, DeviceClasses (last-write)	Generic USB flash drive (Ven_General, Prod_UDisk, serial 6&1bec0f48&0&) registered; F: assigned to it	Confirms device connection and d assignment at the system level; do alone prove a person's physical pr
Drive mapping	15:03:07	E06	MountPoints2 (IEUser's NTUSER.DAT)	Subkey "F" recorded under IEUser's own profile	Directly ties the USB device to IE profile, not just the system general
Command-history evidence	15:06:11-15:06:~30	E03	cmdscan / consoles	cmd.exe (PID 3996) runs ipconfig, cd Downloads, copy secret_file.docx F: → "1 file(s) copied"	Direct evidence the copy comman completed inside Windows; does n independently prove the device wa removed or the file reached its fina destination afterward
Supported conclusion	15:09:06	E01	imageinfo	Memory image captured	Snapshot moment; activity after th unknown

11. Attribution Assessment

Category	Content
Direct observations (what the Volatility output literally shows)	A memory image (Win7SP1x86_23418 profile, validated via pslist/netscan) captured at 2019-01-06 15:09:06 UTC shows: cmd.exe (PID 3996, parent explorer.exe, owned by SID ...-1000/IEUser) executed copy secret_file.docx F:, with the console screen buffer recording "1 file(s) copied." Registry evidence (USBSTOR, MountedDevices, DeviceClasses) confirms a generic USB storage device (serial 6&1bec0f48&0&) was assigned drive letter F:. MountPoints2, read from IEUser's own NTUSER.DAT hive, records a subkey for drive F under that specific profile. ShellBags, read from IEUser's UsrClass.dat, record the Downloads folder and the F:\ volume both being browsed in File Explorer within the same window. Four Chrome processes and one cmd.exe process all ran under the same IEUser SID during this period.
Reasonable inferences (what you conclude, and why)	The close temporal clustering of these artefacts (USB registration at 15:02:54, drive-letter mapping to IEUser's profile at 15:03:07, Downloads folder access at 15:04:40, the copy command itself at approximately 15:06:11-15:06:30, and F:\ volume access at 15:07:03) supports a reasonable inference that a document named secret_file.docx was copied from the IEUser Downloads folder to a USB storage device during an active IEUser session, and that the user then checked the destination drive afterward.
Limitations / alternative explanations (at least three)	1) Shared or multi-person access: IEUser is a local Administrators-group account: process ownership identifies the account the activity ran under, not conclusively a single named human physically present at the keyboard throughout this entire window. 2) Command success within Windows is not proof of a completed real-world transfer: "1 file(s) copied" confirms the Windows copy operation succeeded, but does not by itself confirm the USB device was subsequently removed, the file was not later deleted, or the device left the premises. 3) Registry last-write times record when a key was modified, not a continuous activity log: USBSTOR/MountedDevices/DeviceClasses timestamps mark the most recent write, which may not align exactly to the second with the copy command's actual execution. 4) ShellBag times are folder-view/preference evidence: the Downloads and F:\ entries show these locations were browsed in Explorer, not that any specific file action occurred at that precise moment.

Category	Content
	5) A second account (sshd_server) exists on this machine; while its processes are confined to SSH service infrastructure and show no connection to this sequence, the system is not a single-user machine, which is relevant context for interpreting any future or additional evidence outside the scope of this image.

12. Evidence Log

Evidence ID	Tool and Plugin	Source Record or Path	Observed Fact
E01	imageinfo	N/A (image-level)	Profile Win7SP1x86_23418 validated via pslist/netscan; i captured 2019-01-06 15:09:06 UTC; PST (UTC−8) confir via TimeZoneInformation
E02	pslist	PIDs 2388, 1280, 2380, 2912, 976 (chrome.exe); PID 3996 (cmd.exe)	Chrome launched 15:04:30 (parent explorer.exe 2524); cm launched 15:06:11 (parent explorer.exe 2524)
E03	cmdscan / consoles	conhost.exe PID 3988, attached to cmd.exe PID 3996	Commands: ipconfig; cd Downloads; copy secret_file.doc "1 file(s) copied."
E04	printkey - USBSTOR	ControlSet001\Enum\USB STOR	Disk&Ven_General&Prod_UDisk&Rev_5.00, serial 6&1bec0f48&0&; last-write 15:02:54 UTC
E05	printkey - MountedDevic es	SYSTEM\MountedDevices , \DosDevices\F:	F: mapped to the same USBSTOR serial via matching dev path string; last-write 15:02:54 UTC
E06	Printkey - MountPoints2	IEUser NTUSER.DAT (offset 0x8b5d39b0), Explorer\MountPoints2	Subkey "F" recorded under IEUser's own profile; last-wri 15:03:07 UTC
E07	shellbags	IEUser UsrClass.dat, BagMRU	Downloads folder entry (last-write 15:04:40, folder modif 14:49:40); F:\ volume entry (last-write 15:07:03)

17. Executive Summary

This report examines a Windows memory image (memdumpWin7.mem, SHA-256: 500bc1ba8b5ffa8053b6213d7e7aae0c84c18cce541dc53a55887e6e5099495e) captured at 2019-01-06 15:09:06 UTC and analysed with Volatility 2.6.1 using the Win7SP1x86_23418 profile, validated against pslist and netscan output. The image is confirmed to be Windows 7 SP1 (32-bit), which clarifies the inconsistency between the case scenario's "Windows 10" wording and the supplied file name memdumpWin7.mem.

Process analysis identified cmd.exe (PID 3996, launched 15:06:11 UTC under the IEUser account) issuing the command copy secret_file.docx F:, with the attached console buffer recording the operating system's own confirmation: "1 file(s) copied." Registry analysis of USBSTOR, MountedDevices and DeviceClasses (all last-written 2019-01-06 15:02:54 UTC) consistently identifies a generic USB storage device (serial 6&1bec0f48&0&) assigned to drive letter F:. This device-to-drive mapping is further corroborated under IEUser's own user profile via the MountPoints2 registry key (last-written 15:03:07 UTC) and via ShellBag entries recording both the Downloads folder (14:49:40 / 15:04:40 UTC) and the F:\ volume (15:07:03 UTC) being browsed in File Explorer.

Together, these four independent artefact types, process ownership, system-level USB registry keys, a user-profile-specific device record, and File Explorer folder-view history, are mutually consistent and cluster within an approximately fifteen-minute window. This supports a moderate-confidence assessment that a document named secret_file.docx was copied from the IEUser Downloads folder to a connected USB storage device during an active IEUser session on this workstation.

This conclusion is limited by: the possibility that the IEUser account was accessed by more than one person, as account-level evidence cannot establish a single human's physical presence at the keyboard; the fact that a successful Windows copy operation does not independently confirm the device was removed from the premises or that the file was not later altered or deleted; the nature of registry last-write and ShellBag timestamps, which record artefact modification rather than a continuous activity log; and the presence of a second local account (sshd_server) on this machine, confirming it is not exclusively single-user, though its processes show no connection to the sequence described above.

No further identification or conclusion beyond what is stated above is supported by this memory image alone.